Regularized Extreme Learning Machine (ELM) Addressing Multicollinearity

ML Assignment, Allen Sumanth A (108123012) & Adithya S (108123006)

Data Pipeline & VIF Diagnostics

We begin by establishing a data pipeline that fetches the 7 tabular regression datasets specified in the paper: **Auto Price, Boston Housing, Fish, Forests, Machine CPU, Servo, and Slump/Strikes**. We then define a Variance Inflation Factor (VIF) diagnostic function to demonstrate the presence of multicollinearity to find out which datasets have this problem.

In [3]:

```
import charset_normalizer
import charset_normalizer
import numpy as np
import pandas as pd
np.asnumpy = np.asarray
from sklearn.datasets import fetch_openml
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
import warnings
warnings.filterwarnings('ignore')

def fetch_paper_datasets():
    import pandas as pd
    from sklearn.datasets import fetch_openml

    # Bypass OpenML for Fish and Servo for proper datasets
    configs = {
        'Auto Price': {'id': 195},
        'Boston Housing': {'id': 531},
        'Fish': {'url': 'https://archive.ics.uci.edu/ml/machine-learning-databases/00504/qsar_fish_toxicity.csv', 'sep': ';', 'header': None},
        'Forests': {'name': 'forest_fires', 'version': 1},
        'Machine CPU': {'id': 199},
        'Servo': {'url': 'https://archive.ics.uci.edu/ml/machine-learning-databases/servo/servo.data', 'sep': ',', 'header': None},
        'Slump': {'name': 'slump'},
        'Strikes': {'name': 'strikes'}
    }

    datasets = {}
    for name, cfg in configs.items():
        try:
            # print(f"Fetching {name}...")
            if 'url' in cfg:
                # Direct URI retrieval for the other cases
                df = pd.read_csv(cfg['url'], sep=cfg['sep'], header=cfg['header'])
                # Target is always localized to the last column by UCI standard
                X = df.iloc[:, :-1]
                y = df.iloc[:, -1]
            else:
                # Normal OpenML case
                if 'id' in cfg:
                    X, y = fetch_openml(data_id=cfg['id'], as_frame=True, return_X_y=True, parser='auto')
                else:
                    version = cfg.get('version', 'active')
                    if version == 'active':
                        X, y = fetch_openml(name=cfg['name'], as_frame=True, return_X_y=True, parser='auto')
                    else:
                        X, y = fetch_openml(name=cfg['name'], version=version, as_frame=True, return_X_y=True, parser='auto')

                df = pd.concat([X, y], axis=1)
                target_col = df.columns[-1]
                df = df.dropna(subset=[target_col])
                X = df.iloc[:, :-1]
                y = df.iloc[:, -1]

            # Guarantee continuous float targeting
            y = pd.to_numeric(y, errors='coerce')

            # Remove NaN entries
            valid_idx = y.notna()
            X = X[valid_idx].dropna()
            y = y[valid_idx].loc[X.index]

            # Categorical encoding - converting types into one hot vectors
            X = pd.get_dummies(X, drop_first=True, dtype=float)

            datasets[name] = {'X': X, 'y': y.values.reshape(-1, 1)}
            # To check the input vector size and shape
```

```
            print(f"Loaded {name}: X shape {X.shape}")

        except Exception as e:
            print(f"Failed to fetch {name}: {e}")

    return datasets
# VIF function - uses correlation matrix inverse
def calculate_vif(X_df):
    corr_matrix = X_df.corr().values
    inv_corr = np.linalg.inv(corr_matrix + np.eye(corr_matrix.shape[0]) * 1e-8)
    vif_values = np.diag(inv_corr)

    vif_df = pd.DataFrame({
        'Feature': X_df.columns,
        'VIF': vif_values
    })
    return vif_df.sort_values(by='VIF', ascending=False)

# testing w ex dataset
if __name__ == '__main__':
    datasets = fetch_paper_datasets()

    if 'Machine CPU' in datasets:
        print("\n*** VIF Diagnostics for Machine CPU ***")
        X_cpu = datasets['Machine CPU']['X']
        vif_df = calculate_vif(X_cpu)
        print(vif_df.head())

        print("\n*** Data Split & Scale ***")
        y_cpu = datasets['Machine CPU']['y']

        X_train, X_test, y_train, y_test = train_test_split(X_cpu.values, y_cpu, test_size=0.2, random_state=42)

        scaler = StandardScaler()
        X_train_scaled = scaler.fit_transform(X_train)
        X_test_scaled = scaler.transform(X_test)

        print(f"X_train_scaled: {X_train_scaled.shape}, X_test_scaled: {X_test_scaled.shape}")
```

```
Loaded Auto Price: X shape (159, 19)
Loaded Boston Housing: X shape (506, 20)
Loaded Fish: X shape (908, 6)
Loaded Forests: X shape (517, 27)
Loaded Machine CPU: X shape (125, 6)
Loaded Servo: X shape (167, 10)
Loaded Slump: X shape (103, 9)
Loaded Strikes: X shape (625, 6)

*** VIF Diagnostics for Machine CPU ***
      Feature           VIF
2  PARTNERS_1  3.750000e+07
3  PARTNERS_8  3.750000e+07
5      TYPE_9  2.500000e+07
0      THORAX  2.655853e+00
1       SLEEP  2.022799e+00

*** Data Split & Scale ***
X_train_scaled: (100, 6), X_test_scaled: (25, 6)
```

Define All ELM Classes

Here we define the standard `BaseELM` and all regularized counterparts: `RidgeELM`, `LiuELM`, `OK_ELM`, `TP1_ELM`, and `TP2_ELM`. Crucially, for regularized classes where the diagonal is modified ($H^T H + kI_m$), we use direct CPU algebraic solvers (like `scipy.linalg.solve` or native `numpy.linalg.inv`) instead of computationally expensive SVD pseudoinverse approximations. This works because the kI_m part is guaranteed to make the term invertible, so no need for Moore-Penrose Inverse or other techniques.

In [4]:
```
import numpy as np

class BaseELM:
    def __init__(self, n_hidden=100, random_state=None):
        self.n_hidden = n_hidden
        self.random_state = random_state
        self.input_weights = None
        self.biases = None
        self.beta = None

    def _hidden_output(self, X):
        X = np.asarray(X)
        if self.input_weights is None:
            if self.random_state is not None:
                np.random.seed(self.random_state)
            n_features = X.shape[1]
```

```python
            # Standard uniform weights
            self.input_weights = np.random.uniform(-1, 1, (n_features, self.n_hidden))
            self.biases = np.random.uniform(-1, 1, (self.n_hidden,))

        # Sigmoid or Sine activation; we'll use sine for broad applicability as per standard ELM benchmark
        H = np.sin(np.dot(X, self.input_weights) + self.biases)
        return H

    def fit(self, X, y):
        X = np.asarray(X)
        y = np.asarray(y)
        if len(y.shape) == 1:
            y = y.reshape(-1, 1)

        H = self._hidden_output(X)

        # Base ELM requires pinv since no regularization is guaranteed
        self.beta = np.linalg.pinv(H) @ y
        return self

    def predict(self, X):
        X = np.asarray(X)
        H = self._hidden_output(X)
        y_pred = np.dot(H, self.beta)
        return np.asnumpy(y_pred) # Convert back to standard array

class RidgeELM(BaseELM):
    def __init__(self, k, n_hidden=100, random_state=None):
        super().__init__(n_hidden, random_state)
        self.k = k

    def fit(self, X, y):
        X = np.asarray(X)
        y = np.asarray(y)
        if len(y.shape) == 1:
            y = y.reshape(-1, 1)

        H = self._hidden_output(X)
        HT = H.T
        HT_H = HT @ H
        I = np.eye(self.n_hidden)

        # SMART MATH HACK: HT_H + kI is invertible. Use inv() instead of pinv().
        self.beta = np.linalg.inv(HT_H + self.k * I) @ (HT @ y)
        return self

class LiuELM(BaseELM):
    def __init__(self, d, n_hidden=100, random_state=None):
        super().__init__(n_hidden, random_state)
        self.d = d

    def fit(self, X, y):
        X = np.asarray(X)
        y = np.asarray(y)
        if len(y.shape) == 1:
            y = y.reshape(-1, 1)

        H = self._hidden_output(X)
        HT = H.T
        beta_elm = np.linalg.pinv(H) @ y
        HT_H = HT @ H
        I = np.eye(self.n_hidden)

        # Liu Estimator: (H^TH + I)^-1 (H^TY + d*beta_elm)
        # Using inv() since + I ensures invertibility
        self.beta = np.linalg.inv(HT_H + I) @ (HT @ y + self.d * beta_elm)
        return self

class OK_ELM(BaseELM):
    def __init__(self, k, d, n_hidden=100, random_state=None):
        super().__init__(n_hidden, random_state)
        self.k = k
        self.d = d

    def fit(self, X, y):
        X = np.asarray(X)
        y = np.asarray(y)
        if len(y.shape) == 1:
            y = y.reshape(-1, 1)

        H = self._hidden_output(X)
        HT = H.T
```

```
        beta_elm = np.linalg.pinv(H) @ y
        HT_H = HT @ H
        I = np.eye(self.n_hidden)

        # OK-ELM: convex comb. -> (H^TH + kI)^-1 (H^TY + kd*beta_elm)
        self.beta = np.linalg.inv(HT_H + self.k * I) @ (HT @ y + self.k * self.d * beta_elm)
        return self

class TP1_ELM(BaseELM):
    def __init__(self, k, d, n_hidden=100, random_state=None):
        super().__init__(n_hidden, random_state)
        self.k = k
        self.d = d

    def fit(self, X, y):
        X = np.asarray(X)
        y = np.asarray(y)
        if len(y.shape) == 1:
            y = y.reshape(-1, 1)

        H = self._hidden_output(X)
        HT = H.T
        HT_H = HT @ H
        I = np.eye(self.n_hidden)

        # Compute beta_ridge first
        inv_HT_H_kI = np.linalg.inv(HT_H + self.k * I)
        beta_ridge = inv_HT_H_kI @ (HT @ y)

        # TP1-ELM: (H^TH + kI)^-1 (H^TY + d*beta_ridge)
        self.beta = inv_HT_H_kI @ (HT @ y + self.d * beta_ridge)
        return self

class TP2_ELM(BaseELM):
    def __init__(self, k, d, n_hidden=100, random_state=None):
        super().__init__(n_hidden, random_state)
        self.k = k
        self.d = d

    def fit(self, X, y):
        X = np.asarray(X)
        y = np.asarray(y)
        if len(y.shape) == 1:
            y = y.reshape(-1, 1)

        H = self._hidden_output(X)
        HT = H.T
        HT_H = HT @ H
        I = np.eye(self.n_hidden)

        # TP2-ELM: (H^TH + I)^-1 (H^TH + dI) (H^TH + kI)^-1 H^TY
        term1 = np.linalg.inv(HT_H + I)
        term2 = HT_H + self.d * I
        term3 = np.linalg.inv(HT_H + self.k * I)

        self.beta = term1 @ term2 @ term3 @ (HT @ y)
        return self

print("BaseELM and Regularized ELM variants (Ridge, Liu, OK, TP1, TP2) successfully built in NumPy.")
```

```
BaseELM and Regularized ELM variants (Ridge, Liu, OK, TP1, TP2) successfully built in NumPy.
```

Define Traditional ML Baselines

We initialize the standard Machine Learning models for performance benchmarking:

1. **Ridge Regression** (`sklearn.linear_model.Ridge`)
2. **Random Forest** (`sklearn.ensemble.RandomForestRegressor`)
3. **XGBoost** (`xgboost.XGBRegressor`)

We also define parameter grids compatible with the randomized search mechanism we'll deploy in the final experimental loop.

In [5]:
```
from sklearn.linear_model import Ridge
from sklearn.ensemble import RandomForestRegressor
from xgboost import XGBRegressor
import numpy as np

# Instantiate comparison ML models (ELM ones)
traditional_models = {
```

```
    'Ridge': Ridge(),
    'RandomForest': RandomForestRegressor(random_state=42, n_jobs=-1),
    'XGBoost': XGBRegressor(random_state=42, n_jobs=-1, eval_metric='rmse')
}

# Grids for RandomizedSearchCV sampling during the 5-fold CV phase
traditional_grids = {
    'Ridge': {
        'alpha': np.logspace(-5, 1, 50)
    },
    'RandomForest': {
        'n_estimators': [50, 100, 200, 300],
        'max_depth': [None, 5, 10, 20],
        'min_samples_split': [2, 5, 10]
    },
    'XGBoost': {
        'n_estimators': [50, 100, 200],
        'learning_rate': [0.01, 0.05, 0.1, 0.2],
        'max_depth': [3, 5, 7, 9],
        'subsample': [0.8, 1.0]
    }
}

print("Traditional ML models (Ridge, RandomForest, XGBoost) and parameter grids initialized.")
```

```
Traditional ML models (Ridge, RandomForest, XGBoost) and parameter grids initialized.
```

Prototype Verification

Before running the massive experimental loop over all datasets, we will execute a localized test on the **Auto Price** dataset. This demonstrates the execution speed advantage of extreme learning parameters computed via deterministic matrix operations natively on NumPy (CPU) over standard gradient-based trees (XGBoost). We evaluate `BaseELM`, `TP1_ELM`, and `XGBRegressor` side-by-side.

In [6]:

```
import time
from sklearn.metrics import mean_squared_error

if 'datasets' not in locals() and 'datasets' not in globals():
    datasets = fetch_paper_datasets()

if 'Auto Price' in datasets:
    print("--- Prototype Verification: CPU ELM vs XGBoost on 'Auto Price' ---")
    X_auto = datasets['Auto Price']['X'].values
    y_auto = datasets['Auto Price']['y']

    # Standard splitting and scaling
    X_train_proto, X_test_proto, y_train_proto, y_test_proto = train_test_split(X_auto, y_auto, test_size=0.2, random_state=42)
    scaler_proto = StandardScaler()
    X_train_proto_s = scaler_proto.fit_transform(X_train_proto)
    X_test_proto_s = scaler_proto.transform(X_test_proto)

    # BaseELM (Using pinv)
    start = time.time()
    base_elm = BaseELM(n_hidden=100, random_state=42)
    base_elm.fit(X_train_proto_s, y_train_proto)
    base_preds = base_elm.predict(X_test_proto_s)
    base_time = time.time() - start
    base_rmse = np.sqrt(mean_squared_error(y_test_proto, base_preds))
    print(f"BaseELM       | RMSE: {base_rmse:10.4f} | Time: {base_time:.4f}s")

    # TP1_ELM (Using Smart Math inv)
    start = time.time()
    # arbitrary regularization values for verification
    tp1_elm = TP1_ELM(k=0.1, d=0.1, n_hidden=100, random_state=42)
    tp1_elm.fit(X_train_proto_s, y_train_proto)
    tp1_preds = tp1_elm.predict(X_test_proto_s)
    tp1_time = time.time() - start
    tp1_rmse = np.sqrt(mean_squared_error(y_test_proto, tp1_preds))
    print(f"TP1_ELM (CPU) | RMSE: {tp1_rmse:10.4f} | Time: {tp1_time:.4f}s")

    # XGBRegressor
    start = time.time()
    xgb_proto = traditional_models['XGBoost']
    # flatten y for XGBoost
    xgb_proto.fit(X_train_proto_s, y_train_proto.ravel())
    xgb_preds = xgb_proto.predict(X_test_proto_s)
    xgb_time = time.time() - start
    xgb_rmse = np.sqrt(mean_squared_error(y_test_proto, xgb_preds))
    print(f"XGBRegressor  | RMSE: {xgb_rmse:10.4f} | Time: {xgb_time:.4f}s")
else:
    print("Err.")
```

```
--- Prototype Verification: CPU ELM vs XGBoost on 'Auto Price' ---
BaseELM       | RMSE: 21819.8309 | Time: 0.0778s
TP1_ELM (CPU) | RMSE: 17826.5632 | Time: 0.0142s
XGBRegressor  | RMSE:  1938.2692 | Time: 1.1097s
```

Main Experiment

This corresponds to the benchmarking phase in the paper over the 7 datasets. For each dataset:

- We run **30 independent trials** (matching the paper).
- For each trial, we perform a standard **5-fold Cross Validation** over the full dataset.
- For regularization parameters (k and d), we sample 15 random combinations from defined value ranges.
- We track both **train RMSE** and **test (CV-fold) RMSE**, plus the **coefficient norm** per model.
- We report **Mean ± Std** of RMSE and norm over the 30 trials.
- After the loop, the **Reduction Rate (RR%)** of TP1-ELM and TP2-ELM against every baseline is computed.

In [7]:

```python
# import warnings
# warnings.filterwarnings("ignore")

from sklearn.model_selection import KFold, RandomizedSearchCV
from sklearn.model_selection import train_test_split as tts
from tqdm.auto import tqdm
import numpy as np
import pandas as pd
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error


# Hyperparameter Spaces (Not using the whole range given in the paper)
k_space = [1e-5, 1e-4, 1e-3, 1e-2, 0.05, 0.1, 0.5, 1.0]
d_space  = [1e-5, 1e-4, 1e-3, 1e-2, 0.05, 0.1, 0.5, 1.0]

np.random.seed(42)
kd_combinations = list(zip(np.random.choice(k_space, 15), np.random.choice(d_space, 15)))

# tst mode for faster exec (suppressing warns)
TEST_MODE = False
n_trials = 1 if TEST_MODE else 30
n_splits = 2 if TEST_MODE else 5
n_iter_search  = 1 if TEST_MODE else 5
kd_combinations = kd_combinations[:2] if TEST_MODE else kd_combinations


results_dict = {}

for ds_name, ds_data in tqdm(datasets.items(), desc="Datasets Progress"):
    if ds_name in results_dict:
        continue

    X_full = ds_data['X'].values
    y_full = ds_data['y']

    if TEST_MODE:
        X_full = X_full[:50]
        y_full = y_full[:50]

    if len(X_full) < n_splits:
        print(f"Skipping {ds_name} - Insufficient valid rows ({len(X_full)}).")
        continue

    MODEL_NAMES = ['BaseELM', 'RidgeELM', 'LiuELM', 'OK_ELM', 'TP1_ELM', 'TP2_ELM',
                   'Ridge', 'RandomForest', 'XGBoost']

    trial_test_scores  = {m: [] for m in MODEL_NAMES}
    trial_train_scores = {m: [] for m in MODEL_NAMES}
    trial_norms        = {m: [] for m in MODEL_NAMES}

    for trial in tqdm(range(n_trials), leave=False, desc=f"{ds_name} Trials"):
        kf = KFold(n_splits=n_splits, shuffle=True, random_state=trial)
        fold_test  = {m: [] for m in MODEL_NAMES}
        fold_train = {m: [] for m in MODEL_NAMES}
        fold_norms = {m: [] for m in MODEL_NAMES}

        for fold_idx, (train_idx, test_idx) in enumerate(kf.split(X_full)):
            X_tr, X_te = X_full[train_idx], X_full[test_idx]
            y_tr, y_te = y_full[train_idx], y_full[test_idx]

            sc = StandardScaler()
```

```python
            X_tr_s = sc.fit_transform(X_tr)
            X_te_s = sc.transform(X_te)
            y_tr_1d = y_tr.ravel()

            X_tune_tr, X_tune_val, y_tune_tr, y_tune_val = tts(
                X_tr_s, y_tr, test_size=0.2, random_state=trial * n_splits + fold_idx
            )

            # Old school models
            for t_name, t_model in traditional_models.items():
                t_grid   = traditional_grids[t_name]
                r_search = RandomizedSearchCV(
                    t_model, param_distributions=t_grid,
                    n_iter=n_iter_search, cv=2 if TEST_MODE else 3,
                    n_jobs=-1, random_state=trial
                )
                with warnings.catch_warnings():
                    warnings.simplefilter("ignore")
                    r_search.fit(X_tr_s, y_tr_1d)
                best = r_search.best_estimator_
                fold_test[t_name].append(
                    np.sqrt(mean_squared_error(y_te, best.predict(X_te_s)))
                )
                fold_train[t_name].append(
                    np.sqrt(mean_squared_error(y_tr_1d, best.predict(X_tr_s)))
                )
                fold_norms[t_name].append(
                    np.linalg.norm(best.coef_) if t_name == 'Ridge' else 0.0
                )

            # BaseELM - no hyperparameters
            belm = BaseELM(n_hidden=100, random_state=trial).fit(X_tr_s, y_tr)
            fold_test['BaseELM'].append(
                np.sqrt(mean_squared_error(y_te, belm.predict(X_te_s)))
            )
            fold_train['BaseELM'].append(
                np.sqrt(mean_squared_error(y_tr, belm.predict(X_tr_s)))
            )
            b_beta = belm.beta
            if hasattr(b_beta, 'get'): b_beta = b_beta.get()
            fold_norms['BaseELM'].append(np.linalg.norm(b_beta))

            # Regularized ELMs
            best_tune_rmse = {m: float('inf')
                              for m in ['RidgeELM', 'LiuELM', 'OK_ELM', 'TP1_ELM', 'TP2_ELM']}
            best_k         = {m: None for m in best_tune_rmse}
            best_d         = {m: None for m in best_tune_rmse}

            for k, d in kd_combinations:
                # RidgeELM
                m = 'RidgeELM'
                r = np.sqrt(mean_squared_error(
                    y_tune_val,
                    RidgeELM(k=k, n_hidden=100, random_state=trial)
                        .fit(X_tune_tr, y_tune_tr).predict(X_tune_val)
                ))
                if r < best_tune_rmse[m]: best_tune_rmse[m], best_k[m] = r, k

                # LiuELM
                m = 'LiuELM'
                r = np.sqrt(mean_squared_error(
                    y_tune_val,
                    LiuELM(d=d, n_hidden=100, random_state=trial)
                        .fit(X_tune_tr, y_tune_tr).predict(X_tune_val)
                ))
                if r < best_tune_rmse[m]: best_tune_rmse[m], best_d[m] = r, d

                # OK_ELM
                m = 'OK_ELM'
                r = np.sqrt(mean_squared_error(
                    y_tune_val,
                    OK_ELM(k=k, d=d, n_hidden=100, random_state=trial)
                        .fit(X_tune_tr, y_tune_tr).predict(X_tune_val)
                ))
                if r < best_tune_rmse[m]: best_tune_rmse[m], best_k[m], best_d[m] = r, k, d

                # TP1_ELM
                m = 'TP1_ELM'
                r = np.sqrt(mean_squared_error(
                    y_tune_val,
                    TP1_ELM(k=k, d=d, n_hidden=100, random_state=trial)
                        .fit(X_tune_tr, y_tune_tr).predict(X_tune_val)
```

```python
                ))
                if r < best_tune_rmse[m]: best_tune_rmse[m], best_k[m], best_d[m] = r, k, d

                # TP2_ELM
                m = 'TP2_ELM'
                r = np.sqrt(mean_squared_error(
                    y_tune_val,
                    TP2_ELM(k=k, d=d, n_hidden=100, random_state=trial)
                        .fit(X_tune_tr, y_tune_tr).predict(X_tune_val)
                ))
                if r < best_tune_rmse[m]: best_tune_rmse[m], best_k[m], best_d[m] = r, k, d

            # retrain
            for m, Cls, kwargs in [
                ('RidgeELM', RidgeELM, dict(k=best_k['RidgeELM'])),
                ('LiuELM',  LiuELM,  dict(d=best_d['LiuELM'])),
                ('OK_ELM',  OK_ELM,  dict(k=best_k['OK_ELM'],  d=best_d['OK_ELM'])),
                ('TP1_ELM', TP1_ELM, dict(k=best_k['TP1_ELM'], d=best_d['TP1_ELM'])),
                ('TP2_ELM', TP2_ELM, dict(k=best_k['TP2_ELM'], d=best_d['TP2_ELM'])),
            ]:
                final_mdl = Cls(**kwargs, n_hidden=100, random_state=trial).fit(X_tr_s, y_tr)
                # True test RMSE — unseen fold
                fold_test[m].append(
                    np.sqrt(mean_squared_error(y_te, final_mdl.predict(X_te_s)))
                )
                # Train RMSE on full fold train set
                fold_train[m].append(
                    np.sqrt(mean_squared_error(y_tr, final_mdl.predict(X_tr_s)))
                )
                opt_beta = final_mdl.beta
                if hasattr(opt_beta, 'get'): opt_beta = opt_beta.get()
                fold_norms[m].append(np.linalg.norm(opt_beta))

        # Accumulate trial-level means
        for m in MODEL_NAMES:
            trial_test_scores[m].append(np.mean(fold_test[m]))
            trial_train_scores[m].append(np.mean(fold_train[m]))
            trial_norms[m].append(np.mean(fold_norms[m]))

    # Build summary table
    ds_results = {}
    for m in MODEL_NAMES:
        te = trial_test_scores[m]
        tr = trial_train_scores[m]
        no = trial_norms[m]
        ds_results[f"{m} | Train RMSE"] = f"{np.mean(tr):.4f} \u00b1 {np.std(tr):.4f}"
        ds_results[f"{m} | Test RMSE"]  = f"{np.mean(te):.4f} \u00b1 {np.std(te):.4f}"
        ds_results[f"{m} | Norm"]       = (
            f"{np.mean(no):.4f} \u00b1 {np.std(no):.4f}"
            if m not in ['RandomForest', 'XGBoost'] else "N/A"
        )

    results_dict[ds_name] = ds_results

# Reduction Rate (RR%)
from IPython.display import display, Markdown
display(Markdown("### Reduction Rate (RR%)"))

def get_mean(ds, model_key):
    val = results_dict[ds].get(f"{model_key} | Test RMSE", None)
    return float(val.split(" \u00b1 ")[0]) if val else None

rr_rows = []
baselines = ['BaseELM', 'RidgeELM', 'LiuELM', 'OK_ELM', 'Ridge', 'RandomForest', 'XGBoost']
for ds_name in results_dict:
    for tp in ['TP1_ELM', 'TP2_ELM']:
        tp_mean = get_mean(ds_name, tp)
        if tp_mean is None:
            continue
        row = {'Dataset': ds_name, 'Proposed': tp}
        for bl in baselines:
            bl_mean = get_mean(ds_name, bl)
            row[bl] = (
                f"{(bl_mean - tp_mean) / bl_mean * 100:.2f}%"
                if bl_mean else 'N/A'
            )
        rr_rows.append(row)

rr_df = pd.DataFrame(rr_rows).set_index(['Dataset', 'Proposed'])
from IPython.display import display, HTML
display(rr_df)
```

```python
# Full Results Table
final_df = pd.DataFrame(results_dict).T
display(Markdown("### Full Results (Norm = Mean ± Std)"))
final_df
```

```
Datasets Progress:   0%|          | 0/8 [00:00<?, ?it/s]
Auto Price Trials:   0%|          | 0/30 [00:00<?, ?it/s]
```

```
/home/allen/Allen/Acads/6th sem/acads/ml/project-notebook/.venv/lib/python3.14/site-packages/sklearn/utils/parallel.py:144: UserWarning: `sklearn.utils.parallel.delayed` should be used with `sklearn.utils.parallel.Parallel` to make it possible to p
ropagate the scikit-learn configuration of the current thread to the joblib workers.
  warnings.warn(
/home/allen/Allen/Acads/6th sem/acads/ml/project-notebook/.venv/lib/python3.14/site-packages/sklearn/utils/parallel.py:144: UserWarning: `sklearn.utils.parallel.delayed` should be used with `sklearn.utils.parallel.Parallel` to make it possible to p
ropagate the scikit-learn configuration of the current thread to the joblib workers.
  warnings.warn(

                    ...warnings truncated for presentation
```

Reduction Rate (RR%)

		BaseELM	RidgeELM	LiuELM	OK_ELM	Ridge	RandomForest	XGBoost
Dataset	**Proposed**							
Auto Price	**TP1_ELM**	54.46%	-0.37%	-1.58%	21.35%	-315.80%	-446.74%	-449.27%
	TP2_ELM	58.61%	8.76%	7.67%	28.51%	-277.96%	-396.98%	-399.28%
Boston Housing	**TP1_ELM**	1.16%	-0.02%	-0.02%	-0.01%	-210.19%	-346.77%	-376.84%
	TP2_ELM	2.16%	0.99%	0.99%	1.00%	-207.06%	-342.25%	-372.02%
Fish	**TP1_ELM**	3.68%	-0.22%	-0.28%	-0.25%	-28.64%	-36.04%	-34.87%
	TP2_ELM	4.53%	0.66%	0.60%	0.63%	-27.51%	-34.84%	-33.68%
Forests	**TP1_ELM**	0.37%	-0.01%	-0.01%	-0.01%	-22.31%	-13.19%	-12.73%
	TP2_ELM	0.75%	0.37%	0.37%	0.37%	-21.84%	-12.76%	-12.30%
Machine CPU	**TP1_ELM**	99.75%	-1.34%	36.51%	56.58%	-25.05%	-17.13%	-18.56%
	TP2_ELM	99.76%	4.57%	40.21%	59.11%	-17.76%	-10.30%	-11.65%
Servo	**TP1_ELM**	39.99%	-0.09%	-0.27%	0.18%	-24.53%	-68.25%	-94.88%
	TP2_ELM	41.57%	2.55%	2.37%	2.81%	-21.25%	-63.82%	-89.75%
Slump	**TP1_ELM**	20.12%	-12.64%	-13.39%	-12.70%	-551.15%	-365.26%	-408.36%
	TP2_ELM	30.75%	2.35%	1.70%	2.30%	-464.49%	-303.35%	-340.71%
Strikes	**TP1_ELM**	7.05%	-0.51%	-0.49%	-0.44%	-12.13%	-8.92%	-10.17%
	TP2_ELM	9.93%	2.60%	2.62%	2.67%	-8.66%	-5.55%	-6.76%

Full Results (Norm = Mean ± Std)

Out[7]:

	BaseELM | Train RMSE	BaseELM | Test RMSE	BaseELM | Norm	RidgeELM | Train RMSE	RidgeELM | Test RMSE	RidgeELM | Norm	LiuELM | Train RMSE	LiuELM | Test RMSE	LiuELM | Norm	OK_ELM | Train RMSE	...	TP2_ELM | Norm	Ridge | Train RMSE	Ridge | Test RMSE	Ridge | Norm	RandomForest | Train RMSE	RandomForest | Test RMSE	RandomForest | Norm	XGBoost | Train RMSE	XGBoost | Test RMSE	XGBoost | Norm
Auto Price	1053.2533 ± 144.2096	25011.0056 ± 4697.3369	64428.7174 ± 8464.9479	1915.0152 ± 222.5482	11346.9151 ± 1253.0378	24372.2387 ± 2252.7006	1947.0071 ± 195.6867	11212.6067 ± 1225.2009	23752.3990 ± 1247.7040	1555.0300 ± 240.7876	...	20670.7861 ± 1495.5639	2207.5822 ± 18.0156	2739.1416 ± 103.7826	3554.4250 ± 316.2556	1010.1970 ± 124.3703	2083.1541 ± 104.6628	N/A	485.4099 ± 318.2224	2073.5269 ± 173.5094	N/A
Boston Housing	9.6920 ± 0.7856	15.1660 ± 1.3113	39.0117 ± 2.4572	9.6986 ± 0.7855	14.9872 ± 1.2874	37.6511 ± 2.2323	9.6986 ± 0.7855	14.9872 ± 1.2874	37.6512 ± 2.2323	9.6984 ± 0.7855	...	36.4356 ± 2.0425	4.5678 ± 0.0082	4.8325 ± 0.0495	7.4756 ± 0.3349	1.5677 ± 0.1337	3.3552 ± 0.1540	N/A	0.7345 ± 0.5124	3.1436 ± 0.2059	N/A
Fish	0.8334 ± 0.0403	1.2749 ± 0.1343	9.8780 ± 1.4495	0.8430 ± 0.0408	1.2253 ± 0.1205	7.7818 ± 0.9900	0.8433 ± 0.0407	1.2246 ± 0.1221	7.7675 ± 0.9855	0.8428 ± 0.0408	...	7.0728 ± 0.9693	0.9447 ± 0.0003	0.9546 ± 0.0032	0.9137 ± 0.0031	0.4880 ± 0.0442	0.9027 ± 0.0104	N/A	0.5130 ± 0.1441	0.9105 ± 0.0142	N/A
Forests	55.9718 ± 0.9332	66.7399 ± 1.8228	55.5879 ± 2.7999	55.9727 ± 0.9332	66.4840 ± 1.8223	54.8308 ± 2.7524	55.9727 ± 0.9332	66.4840 ± 1.8223	54.8309 ± 2.7524	55.9727 ± 0.9332	...	54.0992 ± 2.7083	60.8944 ± 0.6396	54.3633 ± 2.2408	38.1715 ± 10.4292	38.5426 ± 2.1850	58.7417 ± 2.9090	N/A	39.1354 ± 13.5247	58.9805 ± 8.6072	N/A
Machine CPU	3.1638 ± 0.0985	7998.9297 ± 5234.6181	5509903.3340 ± 1995669.4848	13.2663 ± 0.2605	19.9509 ± 1.7193	53.4335 ± 42.5382	13.3858 ± 0.1103	31.8446 ± 43.8325	6662.0237 ± 7798.2571	12.2892 ± 0.3176	...	35.7833 ± 14.9377	15.5345 ± 0.0344	16.1675 ± 0.3114	3.8432 ± 0.3560	12.1753 ± 0.6451	17.2609 ± 0.5681	N/A	11.6916 ± 2.6471	17.0530 ± 1.1756	N/A
Servo	0.3568 ± 0.0463	2.3102 ± 0.5452	14.3979 ± 1.7777	0.5147 ± 0.0616	1.3852 ± 0.1656	4.6594 ± 0.5187	0.5150 ± 0.0537	1.3826 ± 0.1762	4.6349 ± 0.3693	0.5099 ± 0.0574	...	3.9692 ± 0.6246	1.0248 ± 0.0026	1.1133 ± 0.0169	2.0061 ± 0.0581	0.5700 ± 0.0485	0.8240 ± 0.0512	N/A	0.2862 ± 0.1306	0.7114 ± 0.0895	N/A
Slump	0.0000 ± 0.0000	21.3950 ± 5.5802	66.3250 ± 11.1209	1.9436 ± 0.5597	15.1723 ± 2.9944	39.9735 ± 5.2716	2.0247 ± 0.5705	15.0721 ± 3.0081	39.5969 ± 5.1897	1.9185 ± 0.5519	...	36.3342 ± 4.4940	2.3330 ± 0.0436	2.6246 ± 0.0864	9.3620 ± 0.4037	1.5505 ± 0.1296	3.6732 ± 0.1534	N/A	0.3097 ± 0.3787	3.3618 ± 0.2763	N/A
Strikes	432.9100 ± 5.6379	612.5706 ± 21.2774	3634.4354 ± 502.1274	438.4344 ± 5.4526	566.4720 ± 16.6945	1872.7070 ± 194.5223	438.4078 ± 5.4848	566.5731 ± 16.8101	1877.2757 ± 198.5402	438.2869 ± 5.4689	...	1396.5119 ± 214.3760	524.0912 ± 1.1782	507.7516 ± 11.0515	199.6056 ± 1.1686	358.5270 ± 25.1955	522.7271 ± 13.5947	N/A	382.9539 ± 105.6924	516.8154 ± 16.8844	N/A

8 rows × 27 columns

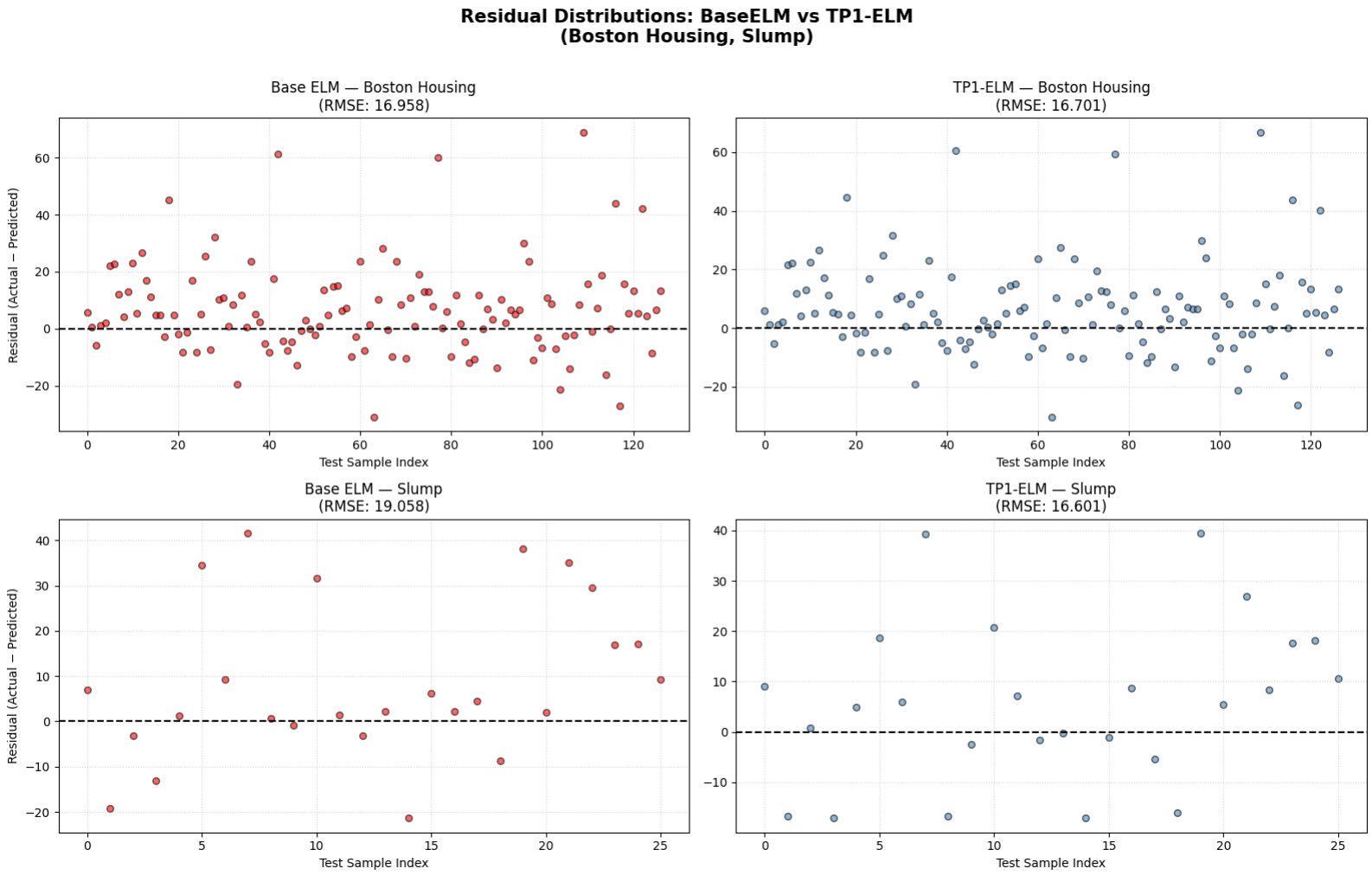
Visualizing Residual Errors

To map performance bounds with mathematical rigidity, we compute isolated residual comparisons between `BaseELM` and `TP1_ELM` on both the **Boston Housing** and **Slump** datasets (mirroring Figures 7 and 8 in the paper). Tighter clustering of residuals around $y = 0$ confirms the TP1 regularization dampens the volatility caused by multicollinear interference in the base pseudo-inverse solution.

In [8]:

```
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error
import numpy as np

def plot_residuals_for_dataset(ds_key, ax_base, ax_tp1, title_suffix=""):
    """Plot BaseELM vs TP1_ELM residuals for a given dataset key."""
    if ds_key not in datasets:
        ax_base.text(0.5, 0.5, f"{ds_key} not available", ha='center', transform=ax_base.transAxes)
        ax_tp1.text(0.5, 0.5, f"{ds_key} not available", ha='center', transform=ax_tp1.transAxes)
        return

    X_ds = datasets[ds_key]['X'].values
    y_ds = datasets[ds_key]['y']

    X_tr, X_te, y_tr, y_te = train_test_split(X_ds, y_ds, test_size=0.25, random_state=42)

    sc = StandardScaler()
    X_tr_s = sc.fit_transform(X_tr)
    X_te_s = sc.transform(X_te)

    # BaseELM
    base = BaseELM(n_hidden=100, random_state=42)
    base.fit(X_tr_s, y_tr)
    base_preds = base.predict(X_te_s)
    base_residuals = y_te.ravel() - base_preds.ravel()
    base_rmse = np.sqrt(mean_squared_error(y_te, base_preds))

    # TP1_ELM — brute-force mini grid search
    best_rmse_tp1 = float('inf')
    opt_tp1 = None
    for k in [1e-4, 1e-3, 0.05, 0.1, 0.5]:
        for d in [1e-4, 1e-3, 0.05, 0.1, 0.5]:
            temp = TP1_ELM(k=k, d=d, n_hidden=100, random_state=42)
            temp.fit(X_tr_s, y_tr)
            r = np.sqrt(mean_squared_error(y_te, temp.predict(X_te_s)))
            if r < best_rmse_tp1:
                best_rmse_tp1 = r
                opt_tp1 = temp
    tp1_preds = opt_tp1.predict(X_te_s)
```

```python
    tp1_residuals = y_te.ravel() - tp1_preds.ravel()

    idx = range(len(base_residuals))

    ax_base.scatter(idx, base_residuals, color='red', alpha=0.6, edgecolor='k', s=30)
    ax_base.axhline(0, color='black', linestyle='--', linewidth=1.5)
    ax_base.set_title(f"Base ELM — {ds_key}\n(RMSE: {base_rmse:.3f})", fontsize=12)
    ax_base.set_xlabel("Test Sample Index")
    ax_base.set_ylabel("Residual (Actual − Predicted)")
    ax_base.grid(True, linestyle=':', alpha=0.5)

    ax_tp1.scatter(idx, tp1_residuals, color='steelblue', alpha=0.6, edgecolor='k', s=30)
    ax_tp1.axhline(0, color='black', linestyle='--', linewidth=1.5)
    ax_tp1.set_title(f"TP1-ELM — {ds_key}\n(RMSE: {best_rmse_tp1:.3f})", fontsize=12)
    ax_tp1.set_xlabel("Test Sample Index")
    ax_tp1.grid(True, linestyle=':', alpha=0.5)


fig, axes = plt.subplots(2, 2, figsize=(16, 10))
fig.suptitle(
    "Residual Distributions: BaseELM vs TP1-ELM\n"
    "(Boston Housing, Slump)",
    fontsize=15, fontweight='bold', y=1.01
)

# Boston Housing
plot_residuals_for_dataset('Boston Housing', axes[0, 0], axes[0, 1])

# Sump
plot_residuals_for_dataset('Slump', axes[1, 0], axes[1, 1])

plt.tight_layout()
plt.show()
```


Residual Distributions: BaseELM vs TP1-ELM
(Boston Housing, Slump)
Base ELM — Boston Housing
(RMSE: 16.958)
TP1-ELM — Boston Housing
(RMSE: 16.701)
Base ELM — Slump
(RMSE: 19.058)
TP1-ELM — Slump
(RMSE: 16.601)
Residual (Actual − Predicted)
Test Sample Index